

Modélisation des données

Projet Paris Basketball — Analyse des revenus par match

Ce document décrit la modélisation mise en place dans le pipeline de données : l'architecture en couches (Data Lake, Staging, Data Warehouse), le choix du schéma en étoile pour la couche finale d'analyse, le détail des tables de faits et de dimensions construites, les décisions de modélisation prises au cours du projet, et les avantages concrets de cette approche.

Document généré le 17/08/2026.

1. Vue d'ensemble du pipeline

Le pipeline est construit en couches successives, chacune ayant un rôle précis. Cette séparation permet de garder les données brutes intactes, de corriger et normaliser progressivement, et de ne construire le modèle d'analyse qu'à la toute dernière étape, une fois les données fiables.

Couche	Dossier	Rôle
Extraction	(en mémoire)	Récupère les données brutes depuis les sources (zip, API...), sans rien écrire sur disque
Load	Data Lake/	Dépose les données brutes telles quelles (JSON, CSV) — première écriture sur disque
Stage	Staging/	Aplatit et normalise les données (tables normalisées, colonnes harmonisées comme ex
Transform	Data Warehouse/	Modélise en schéma en étoile (faits + dimensions) et écrit au format Parquet — la couc

L'orchestrateur (`orchestration/run_pipeline.py`) enchaîne automatiquement Load → Stage → Transform sur le cluster Spark, chaque étape ne démarrant que si la précédente a réussi.

2. Qu'est-ce qu'un schéma en étoile ?

Un schéma en étoile (*star schema*) est une organisation des données pensée pour l'analyse, pas pour la production. Il distingue deux types de tables :

Table de faits

Contient les **mesures** — les chiffres qu'on veut additionner, moyenner, compter. Chaque ligne représente un **événement** qui s'est produit (un billet vendu, une transaction), à un grain précis, avec des clés étrangères qui pointent vers les dimensions.

Table de dimension

Contient le **contexte descriptif** — le qui / quoi / quand / où qui sert à filtrer et regrouper les faits. Relativement statique, peu de lignes comparé aux faits.

Le nom «schéma en étoile» vient de sa forme : une table de faits au centre, entourée de ses dimensions, comme les branches d'une étoile — par opposition à un modèle normalisé (proche de la source, comme notre couche Staging) ou à un schéma en flocon (où les dimensions elles-mêmes sont normalisées en sous-tables).

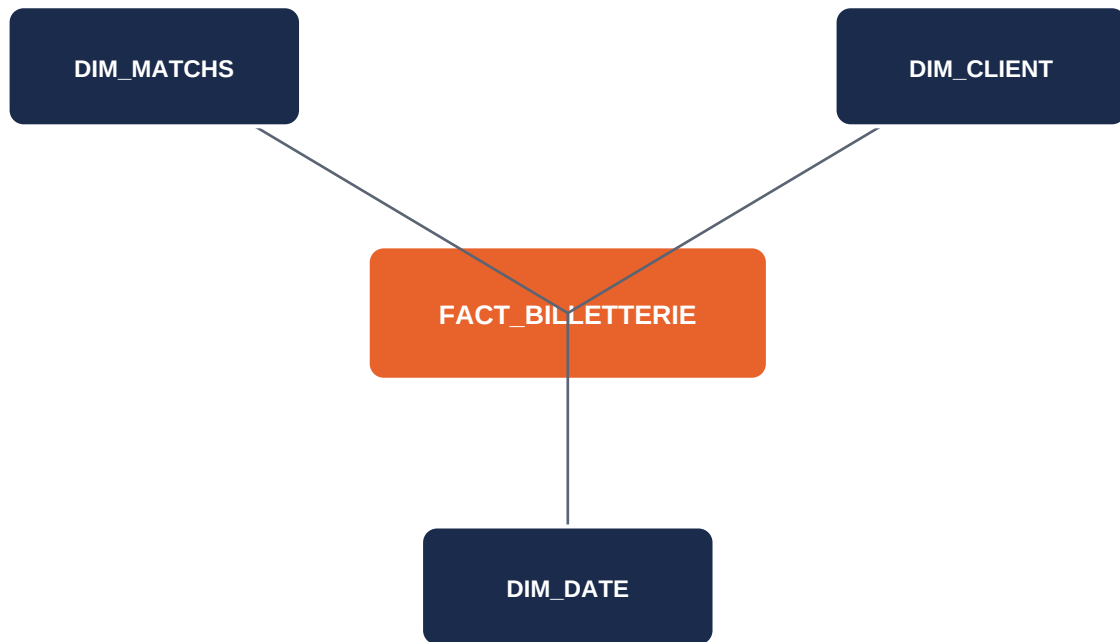


Figure 1 — La table de faits centrale (*fact_billetterie*) porte les montants ; les dimensions autour portent le contexte (quel match, quel client, quel jour). Les autres tables de faits du projet (*fact_buvette*, *fact_boutique*, *fact_commandes*, *fact_paiements*) suivent le même principe, en réutilisant les mêmes dimensions partagées.

3. Les tables de faits

Cinq tables de faits ont été construites, chacune à son propre grain (le niveau de détail d'une ligne), reflétant les processus métier réellement présents dans les données — pas une seule table fusionnée artificiellement.

Table	Grain (1 ligne =)	Mesure	Filtre appliqué
fact_billetterie	un billet vendu	amount	products.is_cancelled = False
fact_buvette	une transaction buvette	montant	aucun
fact_boutique	une ligne de vente boutique	total	aucun (session_id nullable)
fact_commandes	une charge de livraison	amount	aucun
fact_paiements	un paiement	amount	is_cancelled = False

Pourquoi cinq tables séparées plutôt qu'une seule ? Les commandes (orders), produits, billets, frais et paiements n'ont pas le même grain : une commande peut couvrir plusieurs matchs (ex : un abonnement touchant les 41 matchs de la saison). Joindre directement des tables de grains différents avant d'agréger produit un *fan-out* — chaque ligne de la table «large» (ex : une charge au niveau commande) se duplique une fois par ligne de la table «fine» (un billet), ce qui fausse les totaux si on ne fait pas attention. En gardant les faits séparés à leur grain naturel, chaque somme reste exacte.

fact_boutique mérite une précision : les ventes en boutique n'ont aucune référence directe à un match. La colonne `session_id` n'est donc renseignée que pour les clients ayant vu un seul match dans la saison (attribution non ambiguë) ; elle reste vide pour les autres, plutôt que de risquer une attribution incorrecte.

4. Les tables de dimension

Table	Grain	Rôle
dim_matches	un match (session)	Nom, date, lieu, compétition — enrichie par jointure avec venues et competitions.
dim_client	un client (ext_id)	Identifiant client unifié entre billetterie, buvette et boutique.
dim_date	un jour	Calendrier (année, mois, jour de semaine, week-end) pour analyser dans le temps.

dim_matches est la dimension clé de l'analyse «revenu par match» : elle est partagée par `fact_billetterie` (jointure directe via `session_id`) et `fact_buvette` (`session_id` retrouvé par correspondance de date, le `match_id` de la buvette n'ayant pas d'équivalent direct dans le calendrier des sessions). Une dimension partagée entre plusieurs faits est une des forces du schéma en étoile : on compare des revenus de sources différentes sur le même axe.

5. Décisions de modélisation prises

Plusieurs choix de modélisation ont été faits en creusant directement les données plutôt que sur des hypothèses théoriques :

- **Harmonisation de la clé client** — la colonne d'identifiant client externe portait trois noms différents selon la source (`customer_external_id`, `ext_id`, `VENTe_client`) ; elle a été renommée en `ext_id` partout, permettant de construire `dim_client` comme dimension partagée.
- **Filtrage des lignes annulées** — `products.is_cancelled` et `payments.is_cancelled` ont été vérifiés ligne par ligne sur les données réelles : `is_cancelled=True` correspond à des tentatives échouées ou des annulations réelles, pas à du revenu. Ces lignes sont exclues des tables de faits correspondantes.
- **Délimiteur uniforme** — `boutique_ventes_avoirs` utilisait «;» comme séparateur (format français) alors que toutes les autres tables utilisent «.» ; harmonisé en Staging pour que toutes les tables partagent le même délimiteur.
- **Récupération des tables de référence** — les données de calendrier (sessions, venues, events, competitions, teams) étaient présentes dans les fichiers sources bruts mais ignorées par l'extraction initiale ; elles ont été récupérées pour construire `dim_matches`.
- **Dédoublonnage des commandes réexportées** — certaines commandes amendées réapparaissent dans un export ultérieur, créant des identifiants dupliqués (`order_id`, `ticket_id`, `order_product_id`, `charge_id`, `payment_id`). Sans correction, une jointure sur ces identifiants démultiplie les lignes concernées (fan-out). Seule la version du fichier source le plus récent est conservée pour chaque identifiant.
- **Pas de fusion prématurée** — les tables de Staging n'ont volontairement pas été jointes entre elles avant la modélisation, pour garder chaque table à son grain naturel et choisir, table de faits par table de faits, quelles jointures sont réellement pertinentes.

6. Avantages du schéma en étoile

Simplicité des requêtes

Chaque question analytique se traduit par une jointure directe entre une table de faits et une ou plusieurs dimensions, sans chaîne de jointures intermédiaires. «Revenu par match» = `fact_billetterie` + `dim_matches`, point.

Performance analytique

Moins de jointures et des dimensions compactes signifient des agrégations (SUM, COUNT, AVG) rapides, même sur de gros volumes de faits — ici, des centaines de milliers de lignes s'agrègent en quelques secondes.

Compréhensibilité métier

La structure fait / dimension correspond directement à la façon dont on pose une question métier («combien» + «pour qui/quand/où»), ce qui la rend lisible même pour quelqu'un qui ne connaît pas le SQL.

Dimensions partagées et cohérence

dim_matches et dim_client sont utilisées par plusieurs tables de faits (billetterie, buvette, boutique). Toute analyse croisée entre revenus de sources différentes utilise le même référentiel client/match, garantissant des résultats cohérents.

Adapté aux outils de BI et de dataviz

Le format est celui attendu nativement par les outils d'analyse (Streamlit, Power BI, Tableau...) — les tables Parquet typées se chargent directement, sans reparser du texte comme avec du CSV.

Évolutivité

Ajouter une nouvelle dimension (ex: dim_produit) ou une nouvelle table de faits (ex: fact_abonnements) ne casse rien de l'existant — le schéma s'étend sans réécrire les tables déjà en place.

Traçabilité des choix de granularité

Séparer les faits par grain (billet, transaction, paiement, charge) évite les erreurs d'agrégation par fan-out identifiées et corrigées pendant la construction de ce pipeline.

7. Usage prévu

Ce schéma en étoile, écrit au format Parquet dans Data Warehouse/Rev Paris Basketball/, est la couche de données destinée à l'analyse «revenu par match» et, à terme, à la prédiction de revenus pour les premiers matchs de la saison suivante, ainsi qu'aux démonstrations analytiques via Streamlit. La régénération est automatique et quotidienne via le pipeline orchestré.